Le Modulith dans Kubernetes

Déployer le Modulith dans kind, comprendre les pods, externaliser la configuration, passer à Postgres

Objectif de la journée

Aujourd'hui, vous allez faire tourner le Modulith `demo-modulith-sb4` dans un cluster Kubernetes local, **kind** (*Kubernetes in Docker*). Vous allez :

- créer le cluster et comprendre comment Kubernetes est organisé ;
- fabriquer une image de conteneur du Modulith et la charger dans le cluster ;
- déployer le Modulith, l'exposer, le surveiller avec des sondes, le multiplier ;
- constater pourquoi une application doit être sans état, puis brancher une base Postgres partagée et persistante.

Progression

TP	Titre	Ce que vous obtenez
2.1	Le cluster kind et la philosophie Kubernetes	un cluster `formation` opérationnel, observé dans FreeLens
2.2	L'image du Modulith	une image construite sans Dockerfile, chargée dans le cluster
2.3	Premier déploiement	l'API du Modulith joignable depuis votre poste
2.4	Sondes, cycle de vie, montée en charge	3 pods… et des données incohérentes
2.5	Postgres, Secret et configuration externalisée	des données partagées et persistantes

Prérequis

- **Docker Desktop** démarré, avec au moins **10 Go** de mémoire (*Settings → Resources* ; sous Windows avec WSL2, cette limite se règle dans le fichier `.wslconfig`).
- **kind 0.30** ou plus récent, **kubectl**, **JDK 21**, **FreeLens**, **Bruno**.
- Un terminal **bash** : Linux, macOS, WSL ou Git Bash.
- Le projet `demo-modulith-sb4` sur votre poste.

Conventions

- **Dossier du projet** : sauf indication contraire, les commandes se lancent depuis le dossier `demo-modulith-sb4`. Les manifests Kubernetes seront rangés dans son sous-dossier `k8s/`.
- **Tunnels** : un `kubectl port-forward` occupe un terminal à lui seul et reste bloqué tant que le tunnel est ouvert. Quand le pod visé est remplacé, le tunnel s'arrête : **relancez-le**.
- **Résultats attendus** : les noms de pods, les adresses IP, les durées et les numéros de version exacts varient d'un poste à l'autre. Ce qui doit correspondre, c'est la structure, les noms des objets, les états (`Running`, `1/1`…) et les valeurs métier.

Les cinq idées de Kubernetes à retenir

Idée	Ce qu'elle signifie
Modèle déclaratif et réconciliation	vous décrivez l'état voulu ; des contrôleurs font converger l'état réel en boucle. Tout le reste en découle.
Tout est un objet d'API	l'API server est la seule porte d'entrée ; etcd stocke l'état. kubectl et FreeLens ne sont que des clients.
Labels et selectors	un Service ne connaît pas ses pods : il les **sélectionne** par label.
Namespaces	le périmètre de travail : découverte de services, configuration et droits s'y appliquent.
Pods éphémères	un pod est jetable et son adresse IP change. C'est ce qui rend nécessaire la découverte de services.

Les objets que vous allez manipuler

Objet	En un mot	Pour l'observer
Pod	l'unité d'exécution, éphémère	`kubectl get pods -o wide`, `logs`, `describe`
Deployment / ReplicaSet	N copies voulues, remplacement automatique, mise à jour progressive	`kubectl get deploy,rs`, `scale`, `rollout status`
Service + Endpoints	une adresse stable devant un ensemble de pods	`kubectl get svc,endpointslices`
CoreDNS	traduit le nom d'un Service en adresse IP	`nslookup` depuis un pod
ConfigMap / Secret	configuration hors de l'image (un Secret est encodé en base64, **pas chiffré**)	`kubectl get cm,secret`
Sondes	santé du conteneur (démarrage, vie, disponibilité)	`kubectl describe pod`
PVC / PV	un volume de stockage qui survit au pod	`kubectl get pvc,pv`
Namespace	cloisonnement	`kubectl get ns`, l'option `-n`

TP 2.1 — Le cluster kind et la philosophie Kubernetes

But : disposer d'un cluster Kubernetes local opérationnel, et savoir expliquer le rôle du *control plane*, des nœuds et de kubectl.

Étape 1 — Vérifier vos outils

1. Vérifiez que Docker répond, client **et** serveur :

```
docker version
```

Résultat attendu :

```
Client:
 Version:           28.x.x
 ...
Server: Docker Desktop
 Engine:
  Version:          28.x.x
 ...
```

Contrôle : deux blocs, `Client` **et** `Server`. Si seul `Client` apparaît avec `Cannot connect to the Docker daemon`, démarrez Docker Desktop (sous Windows, vérifiez l'intégration WSL dans *Settings → Resources → WSL Integration*).

2. Vérifiez les autres outils :

```
kind version
kubectl version --client
java -version
```

Résultat attendu :

```
kind v0.3x.x go1.2x.x linux/amd64
Client Version: v1.3x.x
openjdk version "21.0.x" ...
```

Contrôle : kind **0.30 ou plus**, et Java **21**. Si une autre version de Java s'affiche, corrigez votre variable `JAVA_HOME`.

Étape 2 — Comprendre l'organisation de Kubernetes

Avant de taper la moindre commande, retenez ces trois rôles :

- **Le control plane** (le cerveau) : `kube-apiserver`, la **seule** porte d'entrée du cluster ; `etcd`, la base de données qui stocke son état ; `kube-scheduler`, qui choisit le nœud où placer chaque pod ; `kube-controller-manager`, qui fait tourner les **boucles de réconciliation**.
- **Les nœuds** (les muscles) : sur chacun, `kubelet` lance les conteneurs, `kube-proxy` gère le réseau des Services, et un *runtime* (ici containerd) exécute les conteneurs.
- **kubectl** (le client) : il ne fait rien lui-même, il envoie des requêtes à l'API server.

À retenir : vous ne lancez pas de conteneurs à la main. Vous **décrivez l'état voulu** (« je veux 3 copies »), l'API server l'enregistre, et des contrôleurs font **converger en boucle** l'état réel vers cet état voulu. Un pod supprimé est recréé.

Étape 3 — Créer le cluster

1. Créez le fichier `kind-cluster.yaml` dans votre dossier de travail. Il décrit un cluster de trois nœuds : un control plane et deux nœuds de travail.

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

2. Créez le cluster `formation` à partir de ce fichier :

```
kind create cluster --name formation --config kind-cluster.yaml
```

Résultat attendu :

```
Creating cluster "formation" ...
 ✓ Ensuring node image (kindest/node:v1.3x.x)
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
 ✓ Installing CNI
 ✓ Installing StorageClass
 ✓ Joining worker nodes
Set kubectl context to "kind-formation"
```

Contrôle : la ligne `Joining worker nodes` (absente sur un cluster d'un seul nœud) et le contexte `kind-formation`.

> **Attention :** le premier démarrage télécharge l'image des nœuds (environ 1 Go) : comptez quelques minutes. kind n'a pas de réglage de mémoire : le cluster utilise la mémoire allouée à Docker Desktop, qui doit être d'au moins **10 Go**.

3. Vérifiez que kubectl s'adresse bien à ce cluster :

```
kubectl config current-context
```

Résultat attendu :

```
kind-formation
```

Contrôle : exactement `kind-formation`. Le **nom du cluster** est `formation`, le **contexte** kubectl est `kind-formation` : ne les confondez pas. kubectl peut connaître plusieurs clusters ; le contexte courant désigne celui à qui il parle.

Étape 4 — Constater que les nœuds sont des conteneurs Docker

1. Listez les conteneurs Docker de votre poste :

```
docker ps --filter name=formation
```

Résultat attendu :

```
IMAGE                  ...   PORTS                       NAMES
kindest/node:v1.3x.x   ...   127.0.0.1:42531->6443/tcp   formation-control-plane
kindest/node:v1.3x.x   ...                               formation-worker
kindest/node:v1.3x.x   ...                               formation-worker2
```

Contrôle : trois conteneurs : chaque **nœud Kubernetes est un conteneur Docker**. Seul le control plane publie un port (`6443`) : celui de l'API server.

Étape 5 — Explorer le control plane

1. Affichez l'adresse de l'API server et les nœuds :

```
kubectl cluster-info
kubectl get nodes -o wide
```

Résultat attendu :

```
Kubernetes control plane is running at https://127.0.0.1:42531
CoreDNS is running at https://127.0.0.1:42531/api/v1/namespaces/...

NAME                       STATUS   ROLES           AGE   VERSION   ...
formation-control-plane    Ready    control-plane   3m    v1.3x.x   ...
formation-worker           Ready    <none>          2m    v1.3x.x   ...
formation-worker2          Ready    <none>          2m    v1.3x.x   ...
```

Contrôle : 3 nœuds `Ready` : un control plane, deux nœuds de travail (`<none>`).

2. Listez les pods du système :

```
kubectl get pods -n kube-system
```

Résultat attendu :

```
NAME                                          READY   STATUS    RESTARTS   AGE
coredns-674b8bbfcf-2x9qk                      1/1     Running   0          3m
coredns-674b8bbfcf-l7m4d                      1/1     Running   0          3m
etcd-formation-control-plane                  1/1     Running   0          3m
kindnet-4zp8t                                 1/1     Running   0          3m
kindnet-9wkqr                                 1/1     Running   0          3m
kindnet-pj2vx                                 1/1     Running   0          3m
kube-apiserver-formation-control-plane        1/1     Running   0          3m
kube-controller-manager-formation-control-plane   1/1     Running   0          3m
kube-proxy-7n6lb                              1/1     Running   0          3m
kube-proxy-c4xdz                              1/1     Running   0          3m
kube-proxy-q8rmn                              1/1     Running   0          3m
kube-scheduler-formation-control-plane        1/1     Running   0          3m
```

Contrôle : tous en `Running`. Les composants du control plane (`etcd`, `kube-apiserver`, `kube-scheduler`, `kube-controller-manager`) sont **eux-mêmes des pods**. `kube-proxy` et `kindnet` (le réseau) tournent en **trois exemplaires**, un par nœud. `coredns` est le DNS interne du cluster.

3. Regardez pourquoi aucune application n'ira sur le control plane :

```
kubectl describe node formation-control-plane | grep -i taint
```

Résultat attendu :

```
Taints:             node-role.kubernetes.io/control-plane:NoSchedule
```

Contrôle : la **taint** `NoSchedule` interdit d'y placer des pods ordinaires : vos applications iront sur les deux nœuds de travail.

Étape 6 — Entrer dans un nœud

1. Ouvrez un shell dans le premier nœud de travail :

```
docker exec -it formation-worker bash
```

2. Listez les conteneurs gérés par le kubelet de ce nœud, puis ressortez :

```
crictl ps
exit
```

Résultat attendu :

```
CONTAINER      STATE     NAME          ...   POD
e1f2a3b4c5d6   Running   kindnet-cni   ...   kindnet-4zp8t
f2a3b4c5d6e1   Running   kube-proxy    ...   kube-proxy-c4xdz
```

Contrôle : les conteneurs des pods placés sur **ce** nœud. Pas de magie : un runtime de conteneurs (containerd) tourne dans le conteneur-nœud.

Attention : `crictl` sert **uniquement à observer**. Ne créez et ne supprimez jamais de conteneurs avec : c'est le kubelet qui en a la charge, et il recréerait aussitôt ce que vous auriez supprimé.

Étape 7 — Voir le modèle déclaratif en action

1. Créez un Deployment de 2 copies de nginx :

```
kubectl create deployment nginx --image=nginx --replicas=2
kubectl get pods -o wide
```

Résultat attendu :

```
deployment.apps/nginx created
NAME                     READY STATUS    ...   IP           NODE
nginx-5869d7778c-4td2h   1/1   Running   ...   10.244.1.2   formation-worker
nginx-5869d7778c-q9xzl   1/1   Running   ...   10.244.2.2   formation-worker2
```

Contrôle : les deux pods sont répartis sur les **deux nœuds de travail**, jamais sur le control plane.

2. Supprimez l'un des deux pods (reprenez un nom affiché chez vous), puis listez à nouveau :

```
kubectl delete pod nginx-5869d7778c-4td2h
kubectl get pods
```

Résultat attendu :

```
pod "nginx-5869d7778c-4td2h" deleted
NAME                     READY   STATUS    RESTARTS   AGE
nginx-5869d7778c-q9xzl   1/1     Running   0          55s
nginx-5869d7778c-7kp2m   1/1     Running   0          4s
```

Contrôle : il y a **toujours 2 pods** : celui supprimé a été remplacé par un nouveau, au nom différent et très récent. C'est la **réconciliation**.

3. Passez à 4 copies, puis supprimez le Deployment :

```
kubectl scale deployment nginx --replicas=4
kubectl get pods
kubectl delete deployment nginx
```

Contrôle : 4 pods après le `scale`, puis plus aucun après la suppression du Deployment. On ne crée jamais un pod « à la main » : on déclare un Deployment.

Étape 8 — Brancher FreeLens

1. Ouvrez FreeLens. Il lit votre fichier `~/.kube/config` : le cluster `kind-formation` apparaît dans la liste. Connectez-vous-y.
2. Explorez **Nodes**, **Pods** et le namespace **kube-system**. Gardez FreeLens ouvert toute la formation : vous **agirez** avec kubectl et vous **observerez** dans FreeLens.

Bilan du TP 2.1

- Cluster `formation` de 3 nœuds, contexte `kind-formation`, visible dans FreeLens.

- Vous savez expliquer : control plane et nœuds, kubectl qui parle à l'API server, la réconciliation.
- Le cluster reste en place pour toute la suite.

TP 2.2 — L'image du Modulith

But : mettre à jour le projet, puis fabriquer une image de conteneur du Modulith selon les bonnes pratiques, et la charger dans le cluster.

Étape 1 — Mettre le pom à jour

1. Placez-vous dans le projet et ouvrez `pom.xml` :

```
cd demo-modulith-sb4
```

2. Dans le bloc `<parent>`, mettez la version de Spring Boot à `4.1.2` :

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.2</version>
    <relativePath/>
</parent>
```

3. Dans `<properties>`, passez Java en **21** (le projet n'utilise rien de Java 25) et Spring Modulith en **2.1.2** :

```
<properties>
    <java.version>21</java.version>
    <spring-modulith.version>2.1.2</spring-modulith.version>
</properties>
```

4. La dépendance `spring-modulith-starter-jpa` est déclarée **deux fois** : supprimez le second bloc.
5. Repérez la dépendance Lombok. Elle porte `<scope>annotationProcessor</scope>`, qui n'est pas un scope reconnu par Maven. Remplacez cette ligne par `<optional>true</optional>` :

```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

6. Dans `<build><plugins>`, ajoutez le plugin de compilation qui active Lombok (la classe `Catalog` l'utilise) :

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <configuration>
        <annotationProcessorPaths>
            <path>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
            </path>
        </annotationProcessorPaths>
    </configuration>
</plugin>
```

7. Compilez et lancez les tests :

```
./mvnw clean package
```

Résultat attendu :

```
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Contrôle : `BUILD SUCCESS`, aucun test en échec.

Étape 2 — Construire l'image sans Dockerfile

Spring Boot sait construire une image avec les **buildpacks** : pas de Dockerfile à écrire. L'image obtenue est découpée en couches (les reconstructions sont rapides), sa base est tenue à jour, elle s'exécute sans les droits root, et la mémoire de Java y est calculée automatiquement.

1. Dans `pom.xml`, repérez le plugin `spring-boot-maven-plugin` et ajoutez-lui ce bloc `<configuration>` :

```
<configuration>
    <image>
        <name>demo-modulith-sb4:0.0.1</name>
        <env>
            <BP_JVM_VERSION>21</BP_JVM_VERSION>
        </env>
    </image>
</configuration>
```

L'image porte un **numéro de version** (`0.0.1`) et jamais `latest` : c'est indispensable pour la charger dans le cluster (étape 3).

2. Construisez l'image (Docker doit être démarré ; le premier build télécharge les outils et prend quelques minutes) :

```
./mvnw spring-boot:build-image
```

Résultat attendu :

```
[INFO]     [creator]       Using Java version 21 from BP_JVM_VERSION
...
[INFO] Successfully built image 'docker.io/library/demo-modulith-sb4:0.0.1'
[INFO] BUILD SUCCESS
```

Contrôle : `Using Java version 21` et `Successfully built image`.

3. Affichez l'image et ses couches :

```
docker images | grep demo-modulith-sb4
docker history demo-modulith-sb4:0.0.1
```

Résultat attendu :

```
demo-modulith-sb4   0.0.1   3f7a1c9e8b21   46 years ago   287MB

IMAGE          CREATED        CREATED BY              SIZE
3f7a1c9e8b21   46 years ago   Application             ...
<missing>      46 years ago   Spring Boot Loader      ...
<missing>      46 years ago   Dependencies            19MB
...
```

Contrôle : les couches `Dependencies`, `Spring Boot Loader` et `Application` sont distinctes : quand le code change, seule la couche `Application` est reconstruite. La date de création à l'an 1980 est **normale** : elle est fixée volontairement pour que deux builds identiques produisent la même image.

Étape 3 — Charger l'image dans le cluster

L'image est dans le Docker de **votre poste**. Le cluster, lui, a son propre stockage d'images : il ne la voit pas tant que vous ne l'y avez pas copiée.

1. Chargez l'image dans le cluster kind (elle est copiée sur **chaque** nœud) :

```
kind load docker-image demo-modulith-sb4:0.0.1 --name formation
```

Résultat attendu :

```
Image: "demo-modulith-sb4:0.0.1" with ID "sha256:3f7a…" not yet present on node
"formation-worker", loading...
Image: "demo-modulith-sb4:0.0.1" with ID "sha256:3f7a…" not yet present on node
"formation-worker2", loading...
Image: "demo-modulith-sb4:0.0.1" with ID "sha256:3f7a…" not yet present on node
"formation-control-plane", loading...
```

2. Vérifiez sa présence dans le cluster :

```
docker exec formation-worker crictl images | grep demo-modulith
```

Résultat attendu :

```
docker.io/library/demo-modulith-sb4   0.0.1   3f7a1c9e8b21a   287MB
```

Contrôle : l'image est visible dans le cluster. Sinon, le déploiement échouera avec `ErrImageNeverPull` ou `ImagePullBackOff`.

Pour aller plus loin (facultatif)

Démarrer plus vite : CDS et AOT

Ces options déplacent du travail au moment du build pour un démarrage environ deux fois plus rapide. Dans le bloc `<env>` du plugin, ajoutez :

```
<BP_JVM_CDS_ENABLED>true</BP_JVM_CDS_ENABLED>
<BP_SPRING_AOT_ENABLED>true</BP_SPRING_AOT_ENABLED>
```

Reconstruisez, rechargez l'image, et comparez la durée affichée dans la ligne `Started … in X seconds` des logs, avec et sans ces options.

L'alternative : un Dockerfile en plusieurs étapes

Si vous voulez tout contrôler vous-même, créez un fichier `Dockerfile` à la racine du projet :

```
# Étape de build : extraction du jar en couches
FROM eclipse-temurin:21-jre AS builder
WORKDIR /builder
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} application.jar
RUN java -Djarmode=tools -jar application.jar extract --layers --destination extracted

# Étape d'exécution : une instruction COPY par couche
FROM eclipse-temurin:21-jre
WORKDIR /application
COPY --from=builder /builder/extracted/dependencies/ ./
COPY --from=builder /builder/extracted/spring-boot-loader/ ./
COPY --from=builder /builder/extracted/snapshot-dependencies/ ./
COPY --from=builder /builder/extracted/application/ ./
ENTRYPOINT ["java", "-jar", "application.jar"]

./mvnw package -DskipTests
docker build -t demo-modulith-sb4:0.0.1-dockerfile .
kind load docker-image demo-modulith-sb4:0.0.1-dockerfile --name formation
```

Vous gagnez le contrôle total, mais vous devez maintenir vous-même l'image de base, l'utilisateur non root et les bonnes pratiques que les buildpacks vous apportent gratuitement.

Bilan du TP 2.2

- pom à jour : Spring Boot 4.1.2, Java 21, Spring Modulith 2.1.2, Lombok correctement déclaré.
- Image `demo-modulith-sb4:0.0.1` construite par buildpacks, en couches, et chargée dans le cluster.

TP 2.3 — Premier déploiement

But : déployer le Modulith avec des fichiers YAML (on décrit l'état voulu, Kubernetes l'applique), l'exposer par un Service et y accéder depuis votre poste. L'application utilise encore sa base H2 en mémoire.

Étape 1 — Créer le namespace

1. Créez le dossier des manifests :

```
mkdir -p k8s
```

2. Créez le fichier `k8s/namespace.yaml` :

```
apiVersion: v1
kind: Namespace
metadata:
  name: shop
```

3. Appliquez-le et listez les namespaces :

```
kubectl apply -f k8s/namespace.yaml
kubectl get ns
```

Résultat attendu :

```
namespace/shop created
NAME              STATUS   AGE
default           Active   20m
kube-node-lease   Active   20m
kube-public       Active   20m
kube-system          Active   20m
local-path-storage   Active   20m
shop                 Active   2s
```

Contrôle : `shop` est `Active` (`local-path-storage` héberge le fournisseur de volumes de kind, utile au TP 2.5). Toute l'application vivra dans ce namespace : précisez `-n shop` dans chaque commande.

Étape 2 — Déployer le Modulith

1. Créez le fichier `k8s/deployment.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: modulith
  namespace: shop
  labels:
    app: modulith
spec:
  replicas: 1
  selector:
    matchLabels:
      app: modulith          # le Deployment retrouve ses pods par ce label
  template:
    metadata:
      labels:
        app: modulith        # label porté par chaque pod
    spec:
      containers:
        - name: modulith
          image: demo-modulith-sb4:0.0.1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          resources:
```

```
            requests:
              cpu: "250m"
              memory: "512Mi"
            limits:
              memory: "768Mi"
```

Trois points à remarquer : `imagePullPolicy: IfNotPresent` (utiliser l'image déjà chargée, sans aller la chercher sur Internet) ; le `selector` qui doit correspondre aux `labels` du pod ; la limite mémoire, que l'image utilise pour calculer la mémoire de Java.

2. Appliquez-le, puis observez la hiérarchie des objets créés :

```
kubectl apply -f k8s/deployment.yaml
kubectl get deploy,rs,pods -n shop
```

Résultat attendu :

```
deployment.apps/modulith created

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/modulith      0/1     1            0           4s

NAME                                    DESIRED   CURRENT   READY   AGE
replicaset.apps/modulith-7d9c8b6f5c     1         1         0       4s

NAME                                READY   STATUS    RESTARTS   AGE
pod/modulith-7d9c8b6f5c-x2k9p       0/1     Running   0          4s
```

Contrôle : la chaîne **Deployment** → **ReplicaSet** → **Pod**. Relancez la commande après une dizaine de secondes : le pod passe à `1/1`.

3. Suivez les logs du démarrage, puis quittez avec Ctrl-C :

```
kubectl logs -n shop deploy/modulith -f
```

Résultat attendu :

```
 :: Spring Boot ::                (v4.1.2)
... Starting DemoModulithSb4Application using Java 21.0.x ...
... Tomcat started on port 8080 (http)
... Started DemoModulithSb4Application in 6.482 seconds
```

Contrôle : `Started … in X seconds`, sans exception.

Attention : si le pod reste en `ImagePullBackOff`, l'image n'est pas dans le cluster ou son nom ne correspond pas exactement au manifest (TP 2.2, étape 3). S'il est en `CrashLoopBackOff`, lisez les logs du conteneur précédent : `kubectl logs -n shop deploy/modulith --previous`.

Étape 3 — Exposer le Modulith par un Service

Un pod est **jetable** : il peut disparaître, renaître ailleurs, et **son adresse IP change**. On ne l'appelle donc jamais directement. Un **Service** est une adresse **stable** (un nom et une IP fixe) placée devant un groupe de pods. Comme un standard téléphonique, il transmet chaque appel à l'un des pods prêts. Il ne connaît pas ses pods à l'avance : il les **sélectionne par label**.

1. Créez le fichier `k8s/service.yaml` :

```
apiVersion: v1
kind: Service
metadata:
  name: modulith
  namespace: shop
spec:
  selector:
    app: modulith            # transmet aux pods qui portent ce label
  ports:
    - port: 80               # le port du Service
      targetPort: 8080       # le port du conteneur
```

2. Appliquez-le et affichez le Service, puis les adresses des pods qu'il a sélectionnés :

```
kubectl apply -f k8s/service.yaml
kubectl get svc -n shop
kubectl get endpointslices -n shop
```

Résultat attendu :

```
service/modulith created
NAME       TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
modulith   ClusterIP   10.96.142.37   <none>        80/TCP    3s

NAME              ADDRESSTYPE   PORTS   ENDPOINTS    AGE
modulith-abc12    IPv4          8080    10.244.0.7   10s
```

Contrôle : un Service de type `ClusterIP`, et **une** adresse de pod derrière lui. Si la colonne `ENDPOINTS` est vide, aucun pod prêt n'est sélectionné : rien ne répondra.

Ne confondez pas les trois adresses :

Adresse	Exemple	Joignable depuis
IP du pod	`10.244.0.7:8080`	le cluster uniquement ; elle change à chaque nouveau pod
IP du Service	`10.96.142.37:80`	un autre pod du cluster ; stable, mais **pas** depuis votre poste
`localhost:8080`	`http://localhost:8080`	votre poste, **uniquement** pendant un `kubectl port-forward` (étape 5)

3. Vérifiez que le nom du Service est résolu par le DNS du cluster, depuis un pod temporaire :

```
kubectl run -n shop tmp --image=busybox --rm -it --restart=Never -- \
  nslookup modulith.shop.svc.cluster.local
```

Résultat attendu :

```
Server:    10.96.0.10
Name:      modulith.shop.svc.cluster.local
Address:   10.96.142.37
pod "tmp" deleted
```

Contrôle : le nom renvoie l'**IP du Service**. Le nom complet se lit : `modulith` (le Service) `.shop` (le namespace) `.svc.cluster.local` (le domaine du cluster). Depuis un pod du même namespace, `modulith` suffit.

Étape 4 — Ajouter deux lectures au Modulith

Le Modulith ne sait que créer des cafés (`POST /coffees`). Pour observer ce qui se passe, vous ajoutez une lecture de chaque côté : la liste des cafés, et le stock d'un café.

1. Dans `src/main/java/org/formation/catalog/internal/CoffeeEntity.java`, ajoutez ce getter :

```
public int getPrice() {
    return price;
}
```

2. Dans `src/main/java/org/formation/inventory/internal/StockEntity.java`, ajoutez ce getter :

```
public int getQuantity() {
    return quantity;
}
```

3. Dans `src/main/java/org/formation/catalog/Catalog.java`, ajoutez `import java.util.List;` en haut du fichier, puis cette méthode et ce record à l'intérieur de la classe :

```
@Transactional(readOnly = true)
public List<CoffeeView> findAll() {
    return repository.findAll().stream()
            .map(c -> new CoffeeView(c.getId(), c.getName(), c.getPrice()))
            .toList();
}

public record CoffeeView(Long id, String name, int price) {}
```

La vue `CoffeeView` fait partie de l'API du module `catalog` : on n'expose jamais l'entité `CoffeeEntity`.

4. Dans `src/main/java/org/formation/catalog/CatalogController.java`, ajoutez les imports `org.springframework.web.bind.annotation.GetMapping` et `java.util.List`, puis cette méthode :

```
@GetMapping
List<Catalog.CoffeeView> all() {
    return catalog.findAll();
}
```

5. Remplacez le contenu de `src/main/java/org/formation/inventory/internal/StockRepository.java` par :

```
package org.formation.inventory.internal;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

public interface StockRepository extends JpaRepository<StockEntity, Long> {

    Optional<StockEntity> findByCoffeeId(Long coffeeId);
}
```

6. Créez le fichier `src/main/java/org/formation/inventory/InventoryController.java`. Il est placé à la racine du module `inventory`, d'où il a le droit d'utiliser la partie interne de son propre module :

```
package org.formation.inventory;

import org.formation.inventory.internal.StockEntity;
import org.formation.inventory.internal.StockRepository;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
```

```
@RequestMapping("stocks")
class InventoryController {

    private final StockRepository repository;

    InventoryController(StockRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/{coffeeId}")
    int quantity(@PathVariable Long coffeeId) {
        return repository.findByCoffeeId(coffeeId)
                .map(StockEntity::getQuantity)
                .orElse(0);
    }
}
```

7. Vérifiez que les frontières entre modules sont respectées :

```
./mvnw test -Dtest=ModularityTests
```

Contrôle : `BUILD SUCCESS`.

8. Reconstruisez l'image, rechargez-la dans le cluster et redémarrez le pod :

```
./mvnw spring-boot:build-image -DskipTests
kind load docker-image demo-modulith-sb4:0.0.1 --name formation
kubectl rollout restart deployment/modulith -n shop
kubectl rollout status deployment/modulith -n shop
```

Résultat attendu :

```
deployment.apps/modulith restarted
deployment "modulith" successfully rolled out
```

Le tag de l'image ne change pas (`0.0.1`) : c'est `rollout restart` qui force Kubernetes à recréer le pod avec la nouvelle image. Vous referez ces quatre commandes après **chaque** modification du code.

Étape 5 — Accéder à l'API depuis votre poste

Les adresses du pod et du Service sont internes au cluster. Pour y accéder depuis votre poste, vous ouvrez un **tunnel** : votre `localhost:8080` est relié au port 80 du Service, qui transmet au port 8080 du pod.

1. Dans un **nouveau terminal**, ouvrez le tunnel et laissez-le tourner :

```
kubectl port-forward -n shop svc/modulith 8080:80
```

Résultat attendu :

```
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

2. Dans votre terminal principal (ou dans Bruno), créez un café puis lisez les données :

```
curl -s -X POST "http://localhost:8080/coffees?name=Arabica&price=5"; echo
curl -s http://localhost:8080/coffees; echo
curl -s http://localhost:8080/stocks/1; echo
```

Résultat attendu :

```
1
[{"id":1,"name":"Arabica","price":5}]
0
```

Contrôle : le `POST` renvoie l'identifiant `1`, la liste contient l'Arabica, et le stock vaut `0`. Ce `0` prouve que l'événement `CoffeeCreated` a fait créer le stock par le module `inventory`, **dans le même processus**.

Étape 6 — Observer la réconciliation

1. Dans FreeLens, repérez le Deployment, son ReplicaSet, le pod, le Service, ses Endpoints, les logs et les événements.
2. Supprimez le pod et observez sa renaissance (Ctrl-C pour quitter) :

```
kubectl delete pod -n shop -l app=modulith
kubectl get pods -n shop -w
```

Résultat attendu :

```
pod "modulith-7d9c8b6f5c-x2k9p" deleted
NAME                        READY   STATUS              RESTARTS   AGE
modulith-7d9c8b6f5c-9mwtd   0/1     ContainerCreating   0          1s
modulith-7d9c8b6f5c-9mwtd   0/1     Running             0          3s
modulith-7d9c8b6f5c-9mwtd   1/1     Running             0          12s
```

Contrôle : un **nouveau** pod, au nom différent, est créé automatiquement.

3. Relancez le tunnel, puis relisez la liste des cafés :

```
curl -s http://localhost:8080/coffees; echo
```

Résultat attendu :

```
[]
```

Attention : l'Arabica a disparu : les données de la base H2 vivaient dans la **mémoire du pod**, qui a été remplacé. C'est le problème que le TP 2.5 résoudra.

Bilan du TP 2.3

- L'API du Modulith répond depuis votre poste via un tunnel.
- Vous avez vu la chaîne Deployment → ReplicaSet → Pod, et le couple Service / Endpoints.
- Un pod supprimé renaît… sans ses données.

TP 2.4 — Sondes, cycle de vie, montée en charge

But : apprendre à Kubernetes à surveiller la santé du Modulith, constater que seuls les pods **prêts** reçoivent du trafic, mettre à jour sans coupure, puis passer à 3 pods.

Étape 1 — Ajouter Actuator

1. Dans `pom.xml`, ajoutez la dépendance :

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

2. Ajoutez ces lignes à `src/main/resources/application.properties` :

```
# Endpoints de santé pour Kubernetes
management.endpoints.web.exposure.include=health
management.endpoint.health.probes.enabled=true
management.endpoint.health.show-details=always
```

3. Reconstruisez, rechargez, redémarrez :

```
./mvnw spring-boot:build-image -DskipTests
kind load docker-image demo-modulith-sb4:0.0.1 --name formation
kubectl rollout restart deployment/modulith -n shop
kubectl rollout status deployment/modulith -n shop
```

4. Relancez le tunnel, puis interrogez les deux endpoints de santé :

```
curl -s localhost:8080/actuator/health/liveness; echo
curl -s localhost:8080/actuator/health/readiness; echo
```

   Résultat attendu :

```
{"status":"UP"}
{"status":"UP"}
```

   Contrôle : les deux répondent `UP`. Une réponse `404` signifie qu'Actuator n'est pas dans l'image : reprenez la boucle de redéploiement.

Étape 2 — Brancher les sondes

Sonde	La question posée	Si elle échoue
startup	« as-tu fini de démarrer ? »	on attend ; les deux autres sondes ne commencent qu'après sa réussite
liveness	« es-tu bloqué ? »	le conteneur est **redémarré**
readiness	« peux-tu recevoir des requêtes ? »	le pod est **retiré** du Service (plus de trafic)

1. Dans `k8s/deployment.yaml`, ajoutez les trois sondes dans le conteneur, au même niveau que `resources:` :

```
        startupProbe:
          httpGet: { path: /actuator/health/liveness, port: 8080 }
          failureThreshold: 30
          periodSeconds: 5
        livenessProbe:
          httpGet: { path: /actuator/health/liveness, port: 8080 }
          periodSeconds: 10
        readinessProbe:
          httpGet: { path: /actuator/health/readiness, port: 8080 }
          periodSeconds: 5
```

2. Appliquez le fichier, puis affichez la description du pod :

```
kubectl apply -f k8s/deployment.yaml
kubectl describe pod -n shop -l app=modulith
```

Résultat attendu :

```
    Liveness:   http-get http://:8080/actuator/health/liveness ... period=10s ...
    Readiness:  http-get http://:8080/actuator/health/readiness ... period=5s ...
    Startup:    http-get http://:8080/actuator/health/liveness ... #failure=30
...
Events:
  Normal   Pulled     ...  Container image "demo-modulith-sb4:0.0.1"
                           already present on machine
  Warning  Unhealthy  ...  Startup probe failed: ... connection refused
```

Contrôle : les trois sondes sont listées, et l'image était `already present on machine` (aucun téléchargement).

L'avertissement `Startup probe failed … connection refused` est **normal** : au tout début, Java démarre encore et le port 8080 n'est pas ouvert. La sonde de démarrage tolère jusqu'à 30 échecs espacés de 5 secondes (150 secondes) : c'est précisément son rôle. Un vrai problème se traduirait par 30 échecs de suite, puis un redémarrage.

> **Attention :** ne faites **jamais** dépendre la sonde de vie (liveness) d'une ressource externe comme la base de données : au moindre incident sur la base, Kubernetes redémarrerait vos pods en boucle.

Étape 3 — Constater que la disponibilité commande le trafic

1. Redémarrez le pod et observez la colonne `READY` (Ctrl-C pour quitter) :

```
kubectl rollout restart deployment/modulith -n shop
kubectl get pods -n shop -w
```

Résultat attendu :

```
NAME                        READY   STATUS    RESTARTS   AGE
modulith-6f4b9c2a8e-9mwtd   0/1     Running   0          2s
modulith-6f4b9c2a8e-9mwtd   1/1     Running   0          11s
```

Un pod peut être `Running` (le conteneur tourne) sans être `1/1` (l'application n'est pas prête).

2. Provoquez une sonde de disponibilité défaillante : dans `k8s/deployment.yaml`, remplacez le chemin de la `readinessProbe` par un chemin inexistant, `/actuator/health/readinesss` (avec trois **s**), puis appliquez :

```
kubectl apply -f k8s/deployment.yaml
kubectl get pods -n shop -o wide
kubectl get endpointslices -n shop
```

Résultat attendu :

```
NAME                          READY   STATUS    RESTARTS   AGE   IP
modulith-6f4b9c2a8e-9mwtd     1/1     Running   0          5m    10.244.0.9
modulith-5b8d7c6e4f-k2m4n     0/1     Running   0          40s   10.244.0.10

NAME             ADDRESSTYPE   PORTS   ENDPOINTS    AGE
modulith-abc12   IPv4          8080    10.244.0.9   1h
```

Contrôle : le nouveau pod reste à `0/1` et son adresse (`10.244.0.10`) **n'apparaît pas** dans les Endpoints : le Service ne lui envoie aucune requête. L'ancien pod, toujours prêt, continue de servir : Kubernetes ne l'arrête pas tant que son remplaçant n'est pas prêt.

3. Corrigez le chemin (`/actuator/health/readiness`) et appliquez de nouveau le fichier.

À retenir : seuls les pods **prêts** reçoivent du trafic. C'est aussi ce que la découverte de services lira au J3.

Étape 4 — Mettre à jour sans coupure, et revenir en arrière

1. Lancez une mise à jour progressive et suivez-la :

```
kubectl rollout restart deployment/modulith -n shop
kubectl rollout status deployment/modulith -n shop
kubectl get rs -n shop
```

Résultat attendu :

```
deployment "modulith" successfully rolled out
NAME                  DESIRED   CURRENT   READY   AGE
modulith-6f4b9c2a8e   0         0         0       10m
modulith-7c1d2e3f4a   1         1         1       25s
```

Contrôle : un **nouveau** ReplicaSet a remplacé l'ancien, qui reste présent à `0`. Grâce à la sonde de disponibilité, le nouveau pod a été prêt **avant** que l'ancien soit arrêté : aucune coupure.

2. Revenez à la version précédente :

```
kubectl rollout undo deployment/modulith -n shop
```

Résultat attendu :

```
deployment.apps/modulith rolled back
```

Contrôle : l'ancien ReplicaSet est réactivé. En situation réelle, une nouvelle version correspond à un **nouveau tag d'image**, pas à un simple redémarrage.

Étape 5 — Passer à 3 pods

1. Passez à 3 copies et observez les adresses derrière le Service :

```
kubectl scale deployment/modulith -n shop --replicas=3
kubectl get pods -n shop -o wide
kubectl get endpointslices -n shop
```

Résultat attendu :

```
NAME                          READY   STATUS    RESTARTS   AGE   IP
modulith-7c1d2e3f4a-k7n2p     1/1     Running   0          3m    10.244.0.12
modulith-7c1d2e3f4a-w9xqd     1/1     Running   0          18s   10.244.0.13
modulith-7c1d2e3f4a-zp3mt     1/1     Running   0          18s   10.244.0.14

NAME             ...   PORTS   ENDPOINTS
modulith-abc12   ...   8080    10.244.0.12,10.244.0.13,10.244.0.14
```

Contrôle : 3 pods prêts, 3 adresses derrière le Service.

2. Créez un café **depuis l'intérieur** du cluster, en passant par le Service :

```
kubectl run -n shop curl --image=curlimages/curl --rm -it --restart=Never -- \
  sh -c 'curl -s -X POST "http://modulith/coffees?name=Arabica&price=5"'
```

3. Lisez la liste des cafés 10 fois de suite, toujours à travers le Service :

```
kubectl run -n shop curl --image=curlimages/curl --rm -it --restart=Never -- \
  sh -c 'for i in $(seq 1 10); do curl -s http://modulith/coffees; echo; done'
```

Résultat attendu :

```
[{"id":1,"name":"Arabica","price":5}]
[]
[]
[{"id":1,"name":"Arabica","price":5}]
[]
...
```

Contrôle : la liste est **tantôt pleine, tantôt vide**, selon le pod qui répond : chaque pod a **sa propre** base H2 en mémoire. Le café n'existe que dans le pod qui a reçu le `POST`.

Pourquoi passer par un pod `curl` dans le cluster ? Parce qu'un `kubectl port-forward` reste attaché à **un seul** pod : il ne montrerait pas la répartition entre les trois.

À retenir : une application qui garde des données en mémoire, multipliée en plusieurs pods, donne des résultats incohérents. L'application doit être **sans état**, et les données doivent vivre dans une **base partagée** : c'est le TP 2.5.

Bilan du TP 2.4

- Trois sondes en place ; un pod non prêt ne reçoit aucun trafic.
- Mise à jour progressive et retour arrière observés.
- 3 pods, et des données incohérentes entre eux.

TP 2.5 — Postgres, Secret et configuration externalisée

But : déployer une base Postgres dans le cluster, ranger ses identifiants dans un **Secret**, brancher le Modulith dessus, et constater que les 3 pods partagent désormais les mêmes données, qui survivent aux redémarrages.

Étape 1 — Préparer l'application

1. Dans `pom.xml`, ajoutez le pilote Postgres :

```
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

2. Créez le fichier `src/main/resources/application-k8s.properties`. Il ne sera lu que lorsque le profil `k8s` est actif, c'est-à-dire dans le cluster :

```
spring.jpa.hibernate.ddl-auto=update
```

> **Attention :** en local, la base H2 est en mode `create-drop` : le schéma est recréé à chaque démarrage. Sur une base **partagée par 3 pods**, chaque pod qui démarre effacerait les données des autres. Le profil `k8s` passe donc en `update`.

3. Reconstruisez l'image et chargez-la (le déploiement sera redémarré à l'étape 5) :

```
./mvnw spring-boot:build-image -DskipTests
kind load docker-image demo-modulith-sb4:0.0.1 --name formation
```

Étape 2 — Créer le Secret et la ConfigMap

Règle simple : ce qui est **sensible** (utilisateur, mot de passe) va dans un **Secret** ; le reste (nom de la base, URL) va dans une **ConfigMap**.

1. Créez le fichier `k8s/db-secret.yaml` :

```
apiVersion: v1
kind: Secret
metadata: { name: db-credentials, namespace: shop }
type: Opaque
stringData:
  POSTGRES_USER: shop
  POSTGRES_PASSWORD: s3cr3t
```

2. Créez le fichier `k8s/db-config.yaml` :

```
apiVersion: v1
kind: ConfigMap
metadata: { name: db-config, namespace: shop }
data:
  POSTGRES_DB: shop
  SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/shop
```

Dans l'URL, `postgres` est le nom du Service de la base, que vous créerez à l'étape 4 : le DNS du cluster le résoudra.

3. Appliquez les deux fichiers, puis lisez le mot de passe stocké dans le Secret :

```
kubectl apply -f k8s/db-secret.yaml -f k8s/db-config.yaml
kubectl get secret db-credentials -n shop -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d;
echo
```

Résultat attendu :

```
secret/db-credentials created
configmap/db-config created
s3cr3t
```

Contrôle : le mot de passe ressort **en clair** : un Secret est seulement encodé en base64, **pas chiffré**. N'importe qui ayant le droit de lire les Secrets du namespace peut le lire.

Étape 3 — Réserver un volume de stockage

1. Créez le fichier `k8s/postgres-pvc.yaml` :

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: postgres-data, namespace: shop }
spec:
  accessModes: [ ReadWriteOnce ]
  resources: { requests: { storage: 1Gi } }
```

2. Appliquez-le et vérifiez son état :

```
kubectl apply -f k8s/postgres-pvc.yaml
kubectl get pvc -n shop
```

Résultat attendu :

```
NAME            STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
postgres-data   Pending                                      standard       3s
```

Contrôle : le PVC est `Pending`, et c'est **normal** avec kind : sa classe de stockage `standard` attend qu'un pod utilise le volume pour le créer, sur le nœud où ce pod sera placé (`WaitForFirstConsumer`). Il passera à `Bound` à l'étape 4.

Le PVC exprime un besoin (« je veux 1 Go de stockage ») ; la classe de stockage créera automatiquement le volume (PV) correspondant. Sans lui, les données de Postgres disparaîtraient avec son pod.

Étape 4 — Déployer Postgres

1. Créez le fichier `k8s/postgres.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: postgres, namespace: shop }
spec:
  replicas: 1
  selector: { matchLabels: { app: postgres } }
  template:
    metadata: { labels: { app: postgres } }
    spec:
      containers:
        - name: postgres
          image: postgres:16
          envFrom:
            - secretRef: { name: db-credentials }   # POSTGRES_USER, POSTGRES_PASSWORD
            - configMapRef: { name: db-config }      # POSTGRES_DB
          env:
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          ports: [ { containerPort: 5432 } ]
```

```
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
      volumes:
        - name: data
          persistentVolumeClaim: { claimName: postgres-data }
---
apiVersion: v1
kind: Service
metadata: { name: postgres, namespace: shop }
spec:
  selector: { app: postgres }
  ports: [ { port: 5432, targetPort: 5432 } ]
```

`envFrom` injecte toutes les clés du Secret et de la ConfigMap comme variables d'environnement. `PGDATA` place les données dans un **sous-dossier** du volume : Postgres refuse de s'initialiser dans un dossier qui n'est pas vide.

2. Appliquez-le et attendez que la base soit prête (Ctrl-C pour quitter) :

```
kubectl apply -f k8s/postgres.yaml
kubectl get pods -n shop -l app=postgres -w
```

Résultat attendu :

```
NAME                        READY   STATUS              RESTARTS   AGE
postgres-6c9d5f7b8d-h4n2k   0/1     ContainerCreating   0          2s
postgres-6c9d5f7b8d-h4n2k   1/1     Running             0          9s
```

3. Vérifiez que le volume a maintenant été créé :

```
kubectl get pvc,pv -n shop
```

Résultat attendu :

```
NAME                                 STATUS   VOLUME        CAPACITY   ...
persistentvolumeclaim/postgres-data   Bound    pvc-3a8f...   1Gi        ...

NAME                           CAPACITY   STATUS   CLAIM                STORAGECLASS
persistentvolume/pvc-3a8f...   1Gi        Bound    shop/postgres-data   standard
```

Contrôle : le PVC est `Bound` : le volume a été créé dès que le pod Postgres en a eu besoin.

Étape 5 — Brancher le Modulith sur Postgres

1. Dans `k8s/deployment.yaml`, ajoutez ce bloc `env` dans le conteneur, au même niveau que `ports:` :

```
        env:
          - name: SPRING_PROFILES_ACTIVE
            value: k8s
          - name: SPRING_DATASOURCE_URL
            valueFrom:
              configMapKeyRef: { name: db-config, key: SPRING_DATASOURCE_URL }
          - name: SPRING_DATASOURCE_USERNAME
            valueFrom:
              secretKeyRef: { name: db-credentials, key: POSTGRES_USER }
          - name: SPRING_DATASOURCE_PASSWORD
            valueFrom:
              secretKeyRef: { name: db-credentials, key: POSTGRES_PASSWORD }
```

Spring Boot associe automatiquement la variable `SPRING_DATASOURCE_URL` à la propriété `spring.datasource.url`. L'URL vient de la ConfigMap, les identifiants du Secret : **la configuration n'est plus dans l'image**.

2. Appliquez le fichier, attendez le redémarrage, puis vérifiez la connexion :

```
kubectl apply -f k8s/deployment.yaml
kubectl rollout restart deployment/modulith -n shop
kubectl rollout status deployment/modulith -n shop
kubectl logs -n shop deploy/modulith | grep -i hikari
```

Résultat attendu :

```
... HikariPool-1 - Starting...
... HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@...
... HikariPool-1 - Start completed.
```

Contrôle : la connexion est une `PgConnection` : le Modulith parle à Postgres. Le message `no suitable driver` signifierait que le pilote Postgres manque dans l'image (étape 1).

Étape 6 — Vérifier la cohérence et la persistance

1. Avec 3 pods, créez un café puis lisez la liste 10 fois à travers le Service :

```
kubectl scale deployment/modulith -n shop --replicas=3
kubectl rollout status deployment/modulith -n shop
kubectl run -n shop curl --image=curlimages/curl --rm -it --restart=Never -- \
  sh -c 'curl -s -X POST "http://modulith/coffees?name=Arabica&price=5"; echo;
         for i in $(seq 1 10); do curl -s http://modulith/coffees; echo; done'
```

Résultat attendu :

```
1
[{"id":1,"name":"Arabica","price":5}]
[{"id":1,"name":"Arabica","price":5}]
...
[{"id":1,"name":"Arabica","price":5}]
```

Contrôle : la liste est **toujours pleine**, quel que soit le pod qui répond : les 3 pods partagent la même base. Comparez avec l'étape 5 du TP 2.4.

2. Supprimez les pods du Modulith **et** celui de Postgres, attendez qu'ils soient de nouveau prêts, puis relisez la liste :

```
kubectl delete pod -n shop -l app=modulith
kubectl delete pod -n shop -l app=postgres
kubectl get pods -n shop -w

kubectl run -n shop curl --image=curlimages/curl --rm -it --restart=Never -- \
  sh -c 'curl -s http://modulith/coffees; echo'
```

Résultat attendu :

```
[{"id":1,"name":"Arabica","price":5}]
```

Contrôle : l'Arabica est **toujours là** : les données survivent aux redémarrages, grâce au volume.

3. Listez les tables de la base :

```
kubectl exec -it -n shop deploy/postgres -- psql -U shop -d shop -c '\dt'
```

Résultat attendu :

```
 Schema |       Name        | Type  | Owner
--------+-------------------+-------+-------
 public | coffee_entity     | table | shop
 public | event_publication | table | shop
 public | stock_entity      | table | shop
(3 rows)
```

4. Regardez la table `event_publication` :

```
kubectl exec -it -n shop deploy/postgres -- psql -U shop -d shop -c \
  'select status, completion_date, event_type from event_publication;'
```

Résultat attendu :

```
  status   |      completion_date       |             event_type
-----------+----------------------------+------------------------------------
 COMPLETED | 2026-10-30 09:14:22.481+00 | org.formation.catalog.CoffeeCreated
(1 row)
```

Contrôle : une ligne par événement `CoffeeCreated` traité. Spring Modulith y enregistre chaque événement, dans la même transaction que la donnée métier : vous verrez au J3 et au J4 pourquoi c'est précieux.

Bilan du TP 2.5

- 3 pods du Modulith et une base Postgres ; données cohérentes entre les pods et persistantes.
- Configuration hors de l'image : URL dans une ConfigMap, identifiants dans un Secret.
- Table `event_publication` présente dans Postgres.

Récapitulatif du J2

Le Modulith tourne dans Kubernetes : une image construite par buildpacks, un Deployment et un Service, des sondes branchées sur Actuator, une base Postgres sur un volume persistant, une configuration rangée dans une ConfigMap et un Secret.

```
namespace shop
+-- Deployment modulith (x3)  --->  Service modulith   :80 -> :8080
|      sondes startup / liveness / readiness
|      profil k8s, connexion à la base (ConfigMap + Secret)
+-- Deployment postgres (x1)  --->  Service postgres   :5432
|      volume postgres-data
+-- Secret    db-credentials (POSTGRES_USER, POSTGRES_PASSWORD)
+-- ConfigMap db-config      (POSTGRES_DB, SPRING_DATASOURCE_URL)
```

À retenir

- Kubernetes est **déclaratif** : vous décrivez l'état voulu, il le maintient.
- Un Service **sélectionne** ses pods par label ; l'IP d'un pod ne sert jamais directement.
- **Disponibilité (readiness) = trafic**, vie (liveness) = redémarrage ; jamais de dépendance externe sur la sonde de vie.
- Une application multipliée doit être **sans état** ; ses données vont dans une base partagée.
- Un Secret est encodé en base64, **pas chiffré**.
- Après chaque modification : construire l'image, `kind load docker-image`, `kubectl rollout restart`.

Questions de réflexion

- Pourquoi un Service ne connaît-il pas ses pods à l'avance ? Que se passe-t-il quand un pod est recréé ailleurs ?
- Pourquoi le tunnel `port-forward` ne permet-il pas de voir la répartition entre les pods ?
- Que se passerait-il si la sonde de vie vérifiait l'accès à Postgres ?
- Les 3 pods partagent maintenant la même base. Est-ce suffisant pour découper l'application en plusieurs services ? Que faudra-t-il changer ?